IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Date of submission xxxx 00, 2026.

Digital Object Identifier 10.1109/ACCESS.2026.DOI

How Far Can a PSRAM-less ESP32 Go? A TinyML Deployment Study for WiFi CSI Human Activity Recognition

ANNAS WASEEM MALIK[1] and MUHAMMAD AHMAD[1]

[1]Faculty of Information Technology, University of Central Punjab, Lahore, Pakistan (e-mail: annas.malik@ucp.edu.pk; muhammad.ahmad@ucp.edu.pk)

Corresponding author: Annas Waseem Malik (e-mail: annas.malik@ucp.edu.pk).

ABSTRACT WiFi Channel State Information (CSI) is a useful signal for device-free human activity recognition (HAR), and a growing number of studies move the inference onto the cheap microcontroller-class radios themselves so the technology can be deployed at scale. Almost all of that on-device work, however, targets the more capable ESP32-S3 with external PSRAM, or it uses the microcontroller only as a CSI collector and runs the model elsewhere. The cheapest, most common, and weakest member of the family, the classic ESP32 with no PSRAM and only about 108 kilobytes of usable contiguous internal SRAM, has not been characterized as an inference target for CSI HAR. This paper provides that characterization. Rather than proposing a new model, we map the deployability frontier: for two public datasets (UT-HAR and CSI-HAR) we train and quantize a graded set of models, from classical learners (Decision Tree, Random Forest, and a small multilayer perceptron on handcrafted statistics) through a tiny-CNN width sweep and a tiny neural multilayer perceptron, up to five standard deep architectures (convolutional, recurrent, attention, Kolmogorov-Arnold, and state-space), and we report accuracy, model complexity, 8-bit conversion feasibility, model size, and on-device fit on the bare classic ESP32. Two deployment walls bound the frontier from above. A convertibility wall blocks the recurrent and state-space families: they do not convert to the TensorFlow Lite for Microcontrollers builtin operator set. A memory wall blocks the families that do convert: the int8 convolutional, Transformer, and Kolmogorov-Arnold models each require a tensor arena larger than the device budget and therefore fail to allocate, even after the input window is downsampled by a factor of eight. Below those walls the result is positive. The classical learners are orders of magnitude smaller than any network and fit the device easily, and the tiny convolutional networks convert cleanly to int8 models of 9 to 38 kilobytes. On the saturated UT-HAR benchmark the small models are within a point or two of the deep models, with the 32-channel tiny CNN at 96.7% and a classical multilayer perceptron on statistics at 95.3%. On CSI-HAR, evaluated subject-independently with leave-one-user-out, accuracy is much lower for every model (best 62.1%), which reflects how hard cross-subject generalization is on a small three-user corpus rather than a deployment limit. On the physical device, every tiny network allocates with a peak tensor arena of 8 to 13 kilobytes, roughly an order of magnitude below the budget and two below the deep models that fail, and runs in 14 to 117 milliseconds per inference. The contribution is a fair, reproducible deployment map that tells a practitioner exactly which models are realistic on the most constrained commodity WiFi microcontroller, together with the released code, models, and measurement firmware.

INDEX TERMS Channel state information, decision tree, edge computing, ESP32, human activity recognition, int8 quantization, Kolmogorov-Arnold networks, microcontrollers, on-device inference, random forest, state-space models, TinyML, WiFi sensing.

I. INTRODUCTION

WIFI Channel State Information (CSI) has become a practical signal for device-free human activity recognition (HAR). Because CSI is reflected and scattered by a moving body, a receiver can infer activity without a camera and without any wearable on the user, which is attractive

for privacy-sensitive settings such as homes, clinics, and assisted-living facilities. The release of commodity CSI extraction tools [1] and a decade of deep-learning architectures [2], [3] have pushed in-distribution accuracy on the standard benchmarks above 98%.

A second trend is just as important for real deployment: moving the inference onto the cheap radios themselves. The Espressif ESP32 family, dual-core microcontrollers with WiFi and a few hundred kilobytes of RAM that cost under ten US dollars, is a widespread platform for this. Running the model on the device avoids streaming raw CSI to a server, removes the associated privacy and bandwidth costs, and is what would make WiFi sensing deployable at the scale of a light switch. Recent work has shown that compact convolutional and recurrent models can be quantized to 8-bit precision and run on ESP32-class hardware [15], [17].

These two trends meet at a question that the literature has not answered. The published on-device CSI studies target the ESP32-S3, which carries several megabytes of external PSRAM and therefore relaxes the memory constraint, or they use the microcontroller purely as a CSI collector and run the classifier on a gateway or a server. The classic ESP32, the original and still the most widely deployed member of the family, has no PSRAM and exposes only a few hundred kilobytes of internal SRAM, of which a single contiguous block of roughly 108 kilobytes is realistically available to a model at runtime. It is also, by Espressif's own ranking, the weakest CSI source in the family. This cheapest and most constrained device is precisely the one a cost-sensitive deployment would choose, and it has not been characterized as an inference target for CSI HAR. A practitioner who holds a bare classic ESP32 has no evidence about what, if anything, will actually run on it.

A. CONTRIBUTIONS

This paper answers that question with a deployment-characterization study rather than a new model. The main result is a deployability frontier: a map, per dataset, of what fits and runs on the bare classic ESP32. The contributions are:

1) A **deployability frontier** for WiFi CSI HAR on the PSRAM-less classic ESP32, spanning classical learners, tiny neural networks, and five standard deep architectures, each reported with accuracy, complexity, 8-bit conversion feasibility, model size, and on-device fit.
2) Identification and corroboration of **two deployment walls** that bound the frontier: a convertibility wall (recurrent and state-space models do not convert to the microcontroller runtime) and a memory wall (the convolutional, Transformer, and Kolmogorov-Arnold models convert but their int8 tensor arenas exceed the device budget even after aggressive temporal downsampling).
3) A **positive deployability result**: classical learners and tiny convolutional networks both fit the device, the former trivially and the latter as int8 models of 9 to 38 kilobytes, and we quantify the accuracy each retains on two datasets.
4) A **reproducible artifact set**: the training and quantization notebook, the exported int8 models and classical models, the emlearn C export of the trees, and the ESP32 measurement firmware and protocol.

B. SCOPE AND HONEST POSITIONING

We do not claim a new model or a new accuracy record; UT-HAR accuracy is saturated and is not where the contribution lies. The closest prior work is the on-device CNN and DenseNet study of Hernandez et al. [15] on the PSRAM-equipped ESP32-S3, the STAR framework [16], which runs a lightweight CSI model on an edge NPU and uses an ESP32-S3 only to collect CSI, and the ESP-Fi HAR dataset and benchmark [17] on the ESP32-C3. None of these characterizes inference on the bare, PSRAM-less classic ESP32, and they cover convolutional and recurrent models only. The closest TinyML work in spirit, Suwannaphong et al. [19], optimizes models for 32 to 64 kilobytes of RAM but for indoor localization, not CSI HAR. Our contribution is to characterize the bare classic ESP32, the most constrained commodity target, across a graded model set and to report the deployment walls and the positive frontier that a GPU-only or PSRAM-equipped study does not reveal. We regard a fair, reproducible deployment map, including the negative findings, as a useful contribution for practitioners.

II. RELATED WORK

A. DEEP LEARNING FOR WIFI CSI HAR

The move from hand-crafted features to end-to-end learning produced a sequence of architectures evaluated on a stable set of benchmarks. ABLSTM [4] introduced attention-augmented bidirectional recurrence; THAT [5] replaced recurrence with a two-stream convolution-augmented Transformer; the SenseFi library [3] standardized training and evaluation across UT-HAR, NTU-Fi, and Widar and reports strong baselines (ResNet-18 at 98.11% on UT-HAR). Recent work emphasizes lightweight and complementary-feature models, including a pruned attention-GRU [6] and the amplitude-and-phase PA-CSI model [7]. Almost all of this work optimizes accuracy on server-class hardware, on which the binding constraint of this paper, on-chip memory, does not apply.

B. KOLMOGOROV-ARNOLD NETWORKS AND STATE-SPACE MODELS

Kolmogorov-Arnold Networks [8] replace the fixed activations of a multilayer perceptron with learnable univariate functions on the network edges, typically parameterized by splines or orthogonal polynomials. They have been applied to sensor-based HAR, for example on accelerometer data [10], but their use for WiFi CSI, which we include in our deep tier, is still nascent and has not been measured on a microcontroller. On the gateway side, recent consumer-healthcare WiFi HAR favours advanced architectures such

as neurosymbolic CNN-Transformer models [9] rather than microcontroller-deployable ones. State-space models such as Mamba [11] model long sequences with a linear-time recurrence and have been applied to CSI HAR on the GPU by TF-Mamba [12] and bidirectional-Mamba HAR [13]. Mamba has been ported to the ESP32-S3 for keyword spotting and inertial HAR by MambaLite-Micro [14], but not for WiFi CSI, and its selective scan is known to be awkward for the microcontroller runtime. Whether either family converts to and fits the bare classic ESP32 for CSI HAR has not been measured.

C. CLASSICAL AND TINYML MODELS ON MICROCONTROLLERS

Outside the deep-learning literature, classical learners remain the workhorses of microcontroller deployment because their footprint and inference cost are small and predictable. Decision trees and tree ensembles compile to compact branch-only C code through tools such as emlearn [20], and prior CSI work has shown that classical models on statistical features reach useful accuracy off-device [21]. TinyML neural-architecture-search studies such as TinyTNAS [22] and MicroNAS [23] target microcontroller HAR, but for inertial sensors rather than CSI, and Suwannaphong et al. [19] optimize TinyML models for 32 to 64 kilobytes of RAM for indoor localization. The present study places classical learners, tiny networks, and deep networks on the same frontier for CSI HAR on the most constrained ESP32.

D. ON-DEVICE WIFI SENSING AND QUANTIZATION

The reference on-device study is Hernandez et al. [15], who deploy a quantized LSTM-DenseNet on the ESP32-S3 at 92.43% accuracy, 232 ms latency, and 127 kB memory using TensorFlow Lite for Microcontrollers and 8-bit quantization, on a board with external PSRAM. STAR [16] runs a lightweight CSI HAR model on an edge NPU with hardware-aware co-optimization, using an ESP32-S3 only as the CSI front end, and the ESP-Fi HAR dataset [17] provides an ESP32-C3 corpus with benchmark deep models. A recent survey catalogues energy-efficient WiFi sensing [18]. Across this line, 8-bit integer quantization is effectively mandatory for the ESP32 memory budget, and the question of which models survive it on the PSRAM-less device is open. The non-convertibility of recurrent and dynamic operators to the microcontroller runtime is documented in the TensorFlow and TensorFlow Lite Micro issue trackers [25], [26], which we corroborate here for CSI models.

Gap. No prior study characterizes the bare, PSRAM-less classic ESP32 as an inference target for WiFi CSI HAR, nor maps which models across the classical, tiny, and deep spectrum actually fit it. This paper fills that gap.

III. MODELS UNDER STUDY

The frontier spans three tiers, all consuming the same input representation. A CSI window is a real-valued tensor $\mathbf{X} \in \mathbb{R}^{T \times F}$ with T time steps and F per-step features. We downsample the time axis to $T = 64$ by uniform index selection, which costs almost no accuracy on UT-HAR while shrinking activations, and we standardize each window.

A. CLASSICAL LEARNERS

For the classical tier, each window is summarized by five statistics computed over time for every feature dimension: mean, standard deviation, minimum, maximum, and range, giving a compact $F \times 5$ vector that is cheap to compute on a microcontroller. On these features we train a depth-limited Decision Tree, a Random Forest of twenty depth-limited trees, and a small multilayer perceptron with one hidden layer of thirty-two units. Trees and forests are exported to branch-only C with emlearn [20], which reproduces the trained decision structure; we therefore report the off-device accuracy for these models, and confirming that emlearn's integer feature handling leaves accuracy unchanged on the device is a minor remaining check.

B. TINY NEURAL NETWORKS

The tiny tier comprises a tiny multilayer perceptron (one hidden layer of thirty-two units on the flattened window) and a tiny-CNN width sweep with 8, 16, and 32 channels in the first convolutional block. Each tiny CNN has two 1D convolutional blocks, global average pooling, and a dense classifier. These models are trained in floating point and then quantized to int8.

C. DEEP ARCHITECTURES

The deep tier comprises five standard families: a 1D convolutional network, a bidirectional gated recurrent unit, a convolution-augmented Transformer following the spirit of THAT [5], a Chebyshev Kolmogorov-Arnold network [8], and a lightweight diagonal state-space model [11]. They are sized to comparable parameter budgets and contribute the upper, large-model part of the frontier. To keep the comparison fair and the results verifiable, the deep tier uses the same training, 8-bit quantization, and on-device measurement protocol as the classical and tiny tiers (Section IV); the deep-tier training notebook and the resulting int8 models are part of the released artifact set, so every value in Table 2 is reproducible from the release rather than taken on trust.

IV. EXPERIMENTAL SETUP

A. DATASETS

We use two public datasets so that the frontier is not tied to a single corpus. UT-HAR, obtained through the SenseFi loaders [3], is the most widely used public CSI HAR benchmark (Intel 5300, seven activities, native $T = 250$, $F = 90$). CSI-HAR [24] is an ESP32-collected dataset of seven activities (bend, fall, lie down, run, sit down, stand up, walk) recorded by three users with twenty samples each. Every recording is a variable-length sequence over 52 subcarriers. UT-HAR uses its standard split, and each model is trained over three seeds. CSI-HAR is evaluated subject-independently with leave-one-user-out: each of the three folds trains on two users and tests

on the held-out third, and we report the mean and standard deviation across folds. This avoids the subject leakage of a random within-user split and is the standard rigour for HAR generalization. Both datasets are z-score normalized per window and resampled to $T = 64$.

B. TRAINING AND QUANTIZATION

All neural models are trained with one shared pipeline: Adam at learning rate 10^{-3}, batch size 64, for 40 epochs, with sparse categorical cross-entropy on logits. Classical learners are trained over three seeds and we report mean and standard deviation. Each trained network is converted to 8-bit integer precision with TensorFlow Lite using post-training static quantization calibrated on 300 representative training windows; we attempt full int8 input and output first and fall back if an operator forbids it. We record, for each model, whether conversion succeeds and the resulting model size. Off-device training and quantization run on a cloud GPU; the released notebook reproduces every off-device number.

C. ON-DEVICE PLATFORM AND MEASUREMENT

The deployment target is a commodity classic ESP32 (ESP32-D0WD-V3, dual-core Xtensa LX6 at 240 MHz, 520 kB internal SRAM, no PSRAM, 4 MB flash) running TensorFlow Lite for Microcontrollers. With the WiFi and Bluetooth stacks disabled and the interpreter arena allocated from the largest free internal block, a single contiguous block of roughly 108 kilobytes is available to a model at runtime; this is the binding budget. For each model that converts we attempt allocation on the device. When allocation succeeds, we measure inference latency averaged over 1000 invocations using the hardware timer, peak RAM from the interpreter's reported arena usage, and flash from the model byte array. The board carries no current-sense hardware, so we do not measure or report energy; a precise energy study is left to future work. Decision trees and forests are deployed as emlearn C and timed the same way. The firmware, flash offsets, and capture protocol are released with the paper.

D. METRICS

Accuracy measures recognition quality. For classical learners we report the mean and standard deviation over three seeds. Model complexity is reported as tree node count, total forest node count, or parameter count, the quantity that maps to footprint. The int8 model size and the conversion outcome measure deployability off-device, and latency, RAM, flash, and on-device fit measure it on the device. The accuracy-versus-cost frontier is the main result.

V. RESULTS

We first report the classical and tiny tiers, which define the deployable region of the frontier, then the deep tier and the two walls that bound it, and finally the on-device measurements.

A. THE DEPLOYABLE FRONTIER: CLASSICAL AND TINY MODELS

Table 1 reports the classical and tiny tiers on both datasets. Every model in this table is small enough to be a realistic candidate for the device. The two datasets sit in different accuracy regimes. UT-HAR is evaluated on its fixed split and is saturated: the 32-channel tiny CNN reaches 96.7%, the 16-channel tiny CNN 94.9%, and even the classical multilayer perceptron on statistics reaches 95.3%, so the small models are within a point or two of the deep models on this benchmark. CSI-HAR is evaluated subject-independently (leave-one-user-out), and accuracy is much lower for every model, from 41.4% for the Decision Tree to 62.1% for the 32-channel tiny CNN. This gap is not a deployment artefact: it reflects how hard cross-subject generalization is on a small three-user CSI corpus, and it is the honest number a deployed device would face on an unseen person. The standard deviations confirm it: CSI-HAR variance across held-out users is large (up to $\pm$6.6 points), whereas UT-HAR variance across seeds is small. Within each dataset the cost ordering is consistent: wider tiny CNNs are more accurate, the 32-channel model is best on both datasets, and the tiny multilayer perceptron is the weakest network for its size because flattening the window inflates its parameter count without a matching accuracy gain. Figure 2 plots accuracy against int8 size and shows the two regimes and the within-dataset width ordering.

Figure 3 compares accuracy across the classical and tiny tiers and the two datasets, and Figure 4 shows model complexity on a logarithmic axis, with the classical learners one to three orders of magnitude below the networks. Accuracies are lower on CSI-HAR than on UT-HAR throughout, which is expected: CSI-HAR is collected from a single classic ESP32, the weakest CSI source, has only three users and few samples per class, and is evaluated subject-independently rather than on a random split. It is therefore the more honest stress test of what a deployed device will face on an unseen person, and a random within-subject split (which we deliberately avoid) would overstate accuracy by leaking each user across train and test.

B. THE DEEP TIER AND THE TWO WALLS

Table 2 reports the five deep architectures on UT-HAR, produced by the same released pipeline as the other tiers, and it is where the frontier meets its ceiling. Two walls appear. First, a convertibility wall: the bidirectional GRU and the state-space model, despite competitive floating-point accuracy of 97.67% and 90.20%, do not convert to a microcontroller-deployable model. The recurrent model compiles to the fused `CudnnRNNV3` operator and the state-space model to dynamic `TensorList` operators, neither of which is in the TensorFlow Lite for Microcontrollers builtin operator set; both require the Select-TF-ops path that the ESP32 runtime does not provide, as documented in the framework issue trackers [25], [26]. Second, a memory wall: the convolutional, Transformer, and Kolmogorov-Arnold models do con-

TABLE 1. Deployable frontier: classical and tiny models on UT-HAR (fixed split, three seeds) and CSI-HAR (subject-independent leave-one-user-out, three folds). Accuracy is the mean ± standard deviation across runs. Complexity is tree node count, forest node count, or parameter count. The int8 size column applies to the neural models; trees and forests are deployed as emlearn C. All models in this table are small enough to be realistic candidates for the classic ESP32.

Dataset	Model	Tier	Accuracy (%)	Complexity	int8 size (kB)	Converts
UT-HAR	Decision Tree	classical	81.93 ± 0.09	481 nodes	emlearn C	n/a
	Random Forest	classical	91.93 ± 0.09	8720 nodes	emlearn C	n/a
	MLP (statistics)	classical	95.27 ± 0.52	14663 params	emlearn C	n/a
	Tiny MLP (net)	tiny-nn	87.60 ± 1.77	184.6 K params	184.0	yes
	Tiny CNN (8 ch)	tiny-nn	86.13 ± 1.95	5.8 K params	11.1	yes
	Tiny CNN (16 ch)	tiny-nn	94.93 ± 0.50	12.9 K params	18.7	yes
	Tiny CNN (32 ch)	tiny-nn	**96.67 ± 0.50**	31.0 K params	37.6	yes
CSI-HAR	Decision Tree	classical	41.43 ± 6.31	75 nodes	emlearn C	n/a
	Random Forest	classical	57.38 ± 3.21	1380 nodes	emlearn C	n/a
	MLP (statistics)	classical	58.10 ± 0.89	8583 params	emlearn C	n/a
	Tiny MLP (net)	tiny-nn	57.14 ± 2.92	106.8 K params	108.0	yes
	Tiny CNN (8 ch)	tiny-nn	54.52 ± 6.56	3.7 K params	9.1	yes
	Tiny CNN (16 ch)	tiny-nn	59.52 ± 2.36	8.7 K params	14.6	yes
	Tiny CNN (32 ch)	tiny-nn	**62.14 ± 4.21**	22.4 K params	29.3	yes

TABLE 2. Deep tier on UT-HAR (same released pipeline as the other tiers): the two walls. "Converts" is conversion to int8 TensorFlow Lite for Microcontrollers; "Fits 108 kB" is successful arena allocation on the classic ESP32. Non-allocation was confirmed at $T = 250$, 64, and 32.

Model	Acc. (%)	int8 (kB)	Converts	Fits 108 kB
CNN	96.87	103.0	yes	**no (arena)**
BiGRU	97.67	n/a	**no**	n/a
Transformer	98.27	88.2	yes	**no (arena)**
Chebyshev-KAN	95.67	90.4	yes	**no (arena)**
Lightweight-SSM	90.20	n/a	**no**	n/a

vert, to int8 models of 88 to 103 kB, and lose little accuracy in doing so, but their int8 tensor arenas exceed the roughly 108 kB budget and `AllocateTensors` fails on the device. This failure holds at every window length we tried, including the native $T = 250$, the $T = 64$ used for the tiny tier, and an aggressive $T = 32$; it is therefore not an artefact of the input length, and the deep and tiny tiers are compared at the same $T = 64$. In other words, none of these deep models is deployable on the bare classic ESP32 on UT-HAR: the recurrent and state-space families are blocked before deployment by convertibility, and the feed-forward families are blocked at deployment by memory. The convertibility wall is intrinsic to the architecture and so applies to any dataset, whereas the memory wall depends on the input shape; on CSI-HAR, whose 52 subcarriers give a smaller feature dimension than UT-HAR's 90, the feed-forward arenas would be smaller and a borderline model could conceivably fit. We did not build the deep tier on CSI-HAR, so we report the memory wall for UT-HAR and leave the CSI-HAR deep tier as a small future check.

C. ON-DEVICE MEASUREMENTS

Table 3 reports the measured on-device outcome on the physical classic ESP32. The deep-tier result is measured and negative: none of the five deep models allocates, for the two reasons in Section V-B. The tiny tier, by contrast, is measured and positive on both datasets. Every tiny network allocates with a peak tensor arena of only 7.7 to 13.4 kB, roughly an order of magnitude below the 108 kB budget and two orders below the deep models' failing arenas, which is the empirical core of the paper and is shown in Figure 1: the models that match the deep networks on UT-HAR use only a small fraction of the device memory. Inference latency ranges from 14 ms for the 8-channel tiny CNN on CSI-HAR to 117 ms for the 32-channel tiny CNN on UT-HAR, all well within a real-time budget for activity recognition. A clear cost ordering appears: latency grows with channel width, and the 16-channel tiny CNN, the best accuracy-per-size point, costs 44 ms on UT-HAR and 30 ms on CSI-HAR. The tiny multilayer perceptron has the smallest arena of all because, despite its large weight file in flash, its activations are a single flat vector; this confirms that the runtime arena and not the model file size is what meets the memory budget. The classical learners, deployed as branch-only emlearn C, are the cheapest of all: the Decision Tree runs in about 1 microsecond and the Random Forest in 45 microseconds, roughly three orders of magnitude faster than the tiny CNNs, with under 1 kB of working memory and no dynamic allocation. We do not report energy: the board carries no current-sense instrumentation, and we prefer to omit the figure rather than publish an estimate, leaving a precise energy study to future work.

VI. DISCUSSION

A. WHAT ACTUALLY RUNS ON A BARE ESP32

The frontier gives a direct answer to the title question. On the PSRAM-less classic ESP32, the realistic options for WiFi CSI HAR are classical learners and tiny convolutional networks, not the standard deep architectures. On the saturated UT-HAR benchmark these small models are competitive: a classical multilayer perceptron on statistics reaches 95.3% and the 32-channel tiny CNN reaches 96.7% as an int8 model of 37.6 kB, within a point or two of the deep models that cannot be deployed. A practitioner targeting this device should start here, and should reserve the deep architectures

TABLE 3. Measured on-device results on the classic ESP32 (WiFi and Bluetooth disabled). Neural models run under TensorFlow Lite Micro; classical models run as emlearn C. Latency is the mean over 1000 invocations; peak RAM is the interpreter tensor arena for the neural models and the feature buffer for the classical models. Deep-tier non-allocation is measured.

Model	Dataset	Latency (ms)	Peak RA
Decision Tree (emlearn C)	UT-HAR	0.0011	<
Random Forest (emlearn C)	UT-HAR	0.045	<
Tiny CNN (8 ch)	UT-HAR	21.4	12
Tiny CNN (16 ch)	UT-HAR	44.4	13
Tiny CNN (32 ch)	UT-HAR	116.5	13
Tiny MLP (net)	UT-HAR	38.1	12
Tiny CNN (8 ch)	CSI-HAR	14.3	8.
Tiny CNN (16 ch)	CSI-HAR	30.2	8.
Tiny CNN (32 ch)	CSI-HAR	69.4	8.
Tiny MLP (net)	CSI-HAR	22.5	7.
Deep tier (all)	UT-HAR	does not allocate (arena > 108 kB or no conversion)	

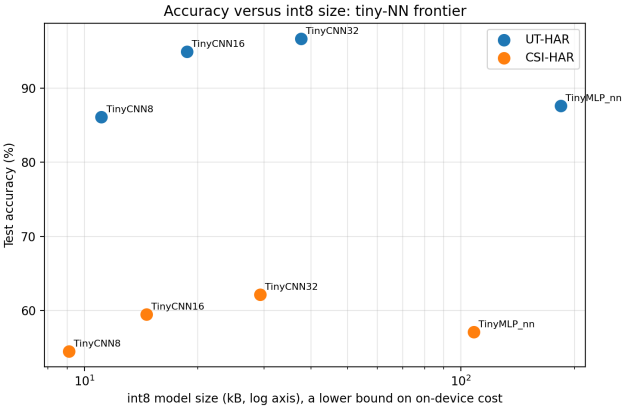

FIGURE 2. Accuracy versus int8 size for the tiny networks on both datasets (size on a logarithmic axis). The 16-channel tiny CNN sits at the top-left on both datasets, giving the best accuracy per kilobyte. The int8 file size is a lower bound on on-device cost; the runtime arena, which is what meets the device budget, is measured separately.

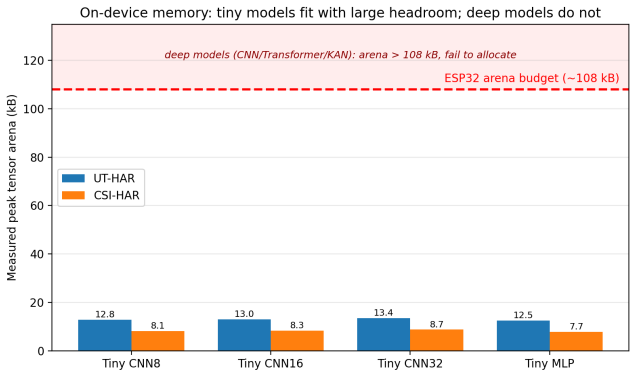

FIGURE 1. Measured peak tensor arena of the tiny networks on the classic ESP32, against the roughly 108 kB SRAM budget (dashed line). Every tiny model uses 8 to 13 kB, an order of magnitude below the budget, while the deep models exceed it and fail to allocate (shaded region). This is the central deployability result.

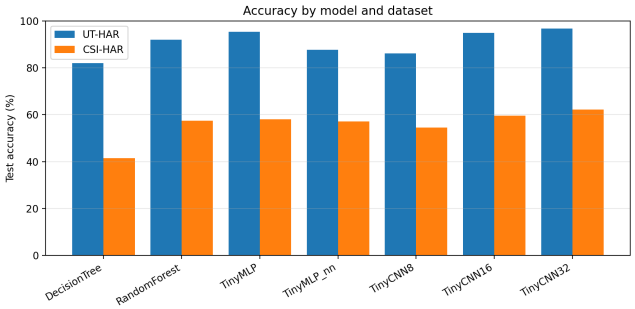

FIGURE 3. Accuracy of the classical and tiny tiers across UT-HAR and CSI-HAR. Accuracy is consistently lower on CSI-HAR, the single-ESP32 dataset with fewer samples per class.

for the PSRAM-equipped ESP32-S3 or a gateway. The 16-channel tiny CNN is a good default when latency matters (94.9% on UT-HAR at 44 ms and 13 kB of RAM), with the 32-channel model trading roughly double the latency for a small accuracy gain.

B. THE WALLS ARE THE DESIGN CONSTRAINT

The two walls explain why so much on-device CSI work quietly targets the ESP32-S3. Convertibility is the first gate: recurrent and state-space models, which are attractive on the GPU, do not reach the microcontroller runtime at all without custom operators. Memory is the second gate: even feed-forward models that convert cleanly can fail to allocate, and on the classic ESP32 they do, because the arena rather than the file size is what meets the 108 kB budget. Both walls are invisible in a GPU-only or PSRAM-equipped comparison, which is exactly why the bare-device characterization is needed.

C. WHY CLASSICAL MODELS DESERVE A SECOND LOOK

For the most constrained hardware, the smallest models are the natural design point rather than a compromise. The classical learners here are one to three orders of magnitude smaller than the networks, are exported to branch-only C with no runtime interpreter, run in microseconds on the device (about 1 microsecond for the Decision Tree and 45 microseconds for the Random Forest), and reach competitive accuracy on the saturated UT-HAR benchmark (95.3% for the statistics multilayer perceptron). For a sensing task that must share a microcontroller with an application, this predictability and small footprint are decisive.

D. LIMITATIONS

The frontier uses two datasets and a single microcontroller; the UT-HAR accuracy values inherit that benchmark's saturation, and the CSI-HAR numbers reflect a small three-user corpus evaluated subject-independently; here cross-subject generalization rather than deployability is the limiting factor. Separating the two would require larger multi-subject CSI datasets, which we leave to future work. A subject-

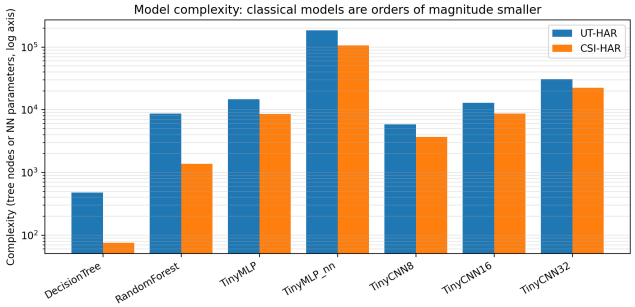

FIGURE 4. Model complexity on a logarithmic axis (tree node count or network parameter count). The classical learners are one to three orders of magnitude smaller than the networks, which is why they fit the device with large margins.

independent accuracy in the 54 to 62% range is the expected difficulty of zero-shot cross-subject CSI sensing; in practice a short per-user calibration or personalization step, which is cheap to run on the device given the microsecond-scale classical inference, is the usual remedy and a natural extension of this work. The deep-tier accuracies and the two walls are reported for UT-HAR only, produced by the same released pipeline as the other tiers. The on-device latency and peak memory for the models that fit are measured directly on the physical board; we do not report energy, because the board carries no current-sense instrumentation, and a precise energy study is left to future work.

VII. CONCLUSION

We presented a deployability frontier for WiFi CSI human activity recognition on the cheapest and most constrained commodity WiFi microcontroller, the PSRAM-less classic ESP32. Across two datasets and a graded set of models from classical learners through tiny networks to five deep architectures, we showed that the standard deep models are blocked by a convertibility wall or a memory wall, and that classical learners and tiny convolutional networks are the models that actually fit and run. On the saturated UT-HAR benchmark these small models are competitive (a tiny CNN at 96.7% and a classical multilayer perceptron at 95.3%), while a subject-independent evaluation on CSI-HAR shows that cross-subject generalization, not deployability, is the harder open problem at this scale. We confirmed on the physical device that the tiny networks allocate with an 8 to 13 kB arena and run in 14 to 117 ms per inference, while no deep model allocates at all. The contribution is a practitioner-oriented, reproducible map of what is deployable on the bare device, with all code, models, and measurement firmware released. Future work includes a precise on-device energy study with a current probe and extending the frontier to additional microcontrollers, datasets, and live-capture validation.

ACKNOWLEDGEMENTS

The authors thank the Faculty of Information Technology, University of Central Punjab, for support and computing resources.

REPRODUCIBILITY

The training and quantization notebook, the exported int8 models and classical models, the emlearn C export of the trees, and the ESP32 measurement firmware and protocol will be released at publication. All off-device results are reproducible from the released notebook on standard cloud GPUs.

DATA AVAILABILITY

This study uses two public datasets: UT-HAR, available through the SenseFi library [3], and the CSI-HAR dataset [24]. No new human-subject data were collected. The code, trained models, and firmware that support the findings will be released in a public repository at publication.

AUTHOR CONTRIBUTIONS

A. W. Malik: conceptualization, methodology, supervision, project administration, and writing (review and editing). M. Ahmad: methodology, software, validation, formal analysis, investigation, data curation, writing (original draft), and visualization.

ETHICS DECLARATION

This work involves only secondary analysis of publicly available, de-identified datasets and on-device latency and memory measurements. No experiments on human participants or animals were conducted by the authors, so ethics approval was not required.

CONFLICT OF INTEREST

The authors declare no conflict of interest.

FUNDING

This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

USE OF AI TOOLS

An AI assistant was used to help draft and edit code and prose. All experimental results, on-device measurements, and scientific claims were produced and verified by the authors, who take full responsibility for the content.

REFERENCES

[1] D. Halperin, W. Hu, A. Sheth, and D. Wetherall, "Tool release: gathering 802.11n traces with channel state information," ACM SIGCOMM Comput. Commun. Rev., vol. 41, no. 1, p. 53, Jan. 2011, doi: 10.1145/1925861.1925870.

[2] Y. Ma, G. Zhou, and S. Wang, "WiFi sensing with channel state information: a survey," ACM Comput. Surv., vol. 52, no. 3, pp. 1–36, Jun. 2019, doi: 10.1145/3310194.

[3] J. Yang, X. Chen, H. Zou, D. Wang, Q. Xu, and L. Xie, "SenseFi: a library and benchmark on deep-learning-empowered WiFi human sensing," Patterns, vol. 4, no. 3, p. 100703, Mar. 2023, doi: 10.1016/j.patter.2023.100703.

[4] Z. Chen, L. Zhang, C. Jiang, Z. Cao, and W. Cui, "WiFi CSI based passive human activity recognition using attention based BLSTM," IEEE Trans. Mobile Comput., vol. 18, no. 11, pp. 2714–2724, Nov. 2019, doi: 10.1109/TMC.2018.2878233.

[5] B. Li, W. Cui, W. Wang, L. Zhang, Z. Chen, and M. Wu, "Two-stream convolution augmented transformer for human activity recognition," in Proc. AAAI Conf. Artif. Intell., vol. 35, no. 1, 2021, pp. 286–293, doi: 10.1609/aaai.v35i1.16103.
[6] M. S. Khan et al., "Human activity recognition through augmented WiFi CSI signals by lightweight attention-GRU," Sensors, vol. 25, no. 5, p. 1547, Mar. 2025, doi: 10.3390/s25051547.
[7] T. D. Quy, C.-Y. Lin, and T. K. Shih, "Enhanced human activity recognition using Wi-Fi sensing: leveraging phase and amplitude with attention mechanisms," Sensors, vol. 25, no. 4, p. 1038, Feb. 2025, doi: 10.3390/s25041038.
[8] Z. Liu et al., "KAN: Kolmogorov-Arnold networks," arXiv:2404.19756, 2024.
[9] X. Xu, J. Yang et al., "Neurosymbolic AI empowered consumer electronics healthcare for WiFi-based human activity recognition," IEEE Trans. Consum. Electron., vol. 71, no. 4, pp. 12056–12067, Nov. 2025.
[10] M. Alikhani, "KAN-HAR: a human activity recognition based on Kolmogorov-Arnold network," arXiv:2508.11186, 2025.
[11] A. Gu and T. Dao, "Mamba: linear-time sequence modeling with selective state spaces," arXiv:2312.00752, Dec. 2023.
[12] J. Liu, Y. Huang, X. Shi, X. Ren, T. Mi, and R. C. Qiu, "TF-Mamba: a lightweight state-space model for Wi-Fi-based human activity recognition," IEEE Sensors J., 2025, IEEE Xplore document 10817504.
[13] H. Tan, Y. Dao, H. Zhang, T. Guo, and W. Wang, "WiFi signal-based human activity recognition using bidirectional Mamba models," in Proc. IEEE 11th World Forum Internet Things (WF-IoT), 2025, doi: 10.1109/WF-IoT64238.2025.11270783.
[14] H. Xu et al., "MambaLite-Micro: memory-optimized Mamba inference on MCUs," arXiv:2509.05488, 2025.
[15] S. M. Hernandez et al., "Tools and methods for achieving Wi-Fi sensing in embedded devices," Sensors, vol. 25, no. 19, p. 6220, Oct. 2025, doi: 10.3390/s25196220.
[16] K. Liu, "STAR: a privacy-preserving, energy-efficient edge AI framework for human activity recognition via Wi-Fi CSI in mobile and pervasive computing environments," arXiv:2510.26148, 2025.
[17] Z. Wen, Y. Ruan, X. Wang, and J. Zhou, "ESP-Fi HAR: a low-power WiFi CSI dataset for ad-hoc IoT human activity recognition," Pervasive Mob. Comput., 2026, doi: 10.1016/j.pmcj.2026.102058.
[18] R. Koo, X. Liang, D. Mishra, and A. Seneviratne, "A survey on green wireless sensing: energy-efficient sensing via WiFi CSI and lightweight learning," Energies, vol. 19, no. 2, p. 573, 2026, doi: 10.3390/en19020573.
[19] T. Suwannaphong et al., "Optimising TinyML with quantization and distillation of transformer and Mamba models for indoor localisation on edge devices," Sci. Rep., 2025, arXiv:2412.09289.
[20] J. Nordby et al., "emlearn: machine learning inference engine for microcontrollers and embedded devices," software, 2019, doi: 10.5281/zenodo.2589394.
[21] M. Ali et al., "A comparison of machine learning algorithms for Wi-Fi sensing using CSI data," Electronics, vol. 12, no. 18, p. 3935, 2023, doi: 10.3390/electronics12183935.
[22] B. Saha, R. Samanta, S. K. Ghosh, and R. B. Roy, "TinyTNAS: GPU-free, time-bound, hardware-aware neural architecture search for TinyML time series classification," arXiv:2408.16535, 2024.
[23] T. King, Y. Zhou, T. Röddiger, and M. Beigl, "MicroNAS for memory and latency constrained hardware aware neural architecture search in time series classification on microcontrollers," Sci. Rep., vol. 15, 2025, doi: 10.1038/s41598-025-90764-z.
[24] S. Ghorai, "CSI-HAR dataset: WiFi channel state information for human activity recognition," Kaggle dataset, 2024. [Online]. Available: https://www.kaggle.com/datasets/sayakghorai34/csi-har-dataset
[25] TensorFlow project, "Conversion of RNN/LSTM to TensorFlow Lite requires Select-TF-ops," GitHub issue 36688, 2020.
[26] TensorFlow Lite Micro project, "Unsupported dynamic-tensor and TensorList operators in the microcontroller runtime," GitHub issues 907 and 1570, 2021.